Artificial Neural Networks

Research question : How effectively can Artificial Neural Networks (ANNs) simulate human learning processes ?

Table of contents

Introduction……..3

Key definitions……3

Introduction to ANNs……5

Supervised Learning……8

Forward propagation……8

Introduction to ANNs' training…….11

Backpropagation……..13

Reinforcement Learning……20

Q-Learning……20

Conclusion……22

Bibliography……24

Appendix……25

A.1……25

A.2……29

Introduction:

Artificial Neural Networks (ANNs) are a type of Machine Learning model inspired by the human brain. They're designed to recognize patterns and make decisions by processing data through interconnected layers of "neurons," much like how our brains process information. ANNs are behind many modern technologies, from image recognition requiring Convolutional Neural Networks (CNNs) to memory recall requiring Recurrent Neural Networks (RNNs). This field evolved exponentially since the growth of data thanks to the Internet and recent breakthroughs in algorithms. This is because, ANNs need a tremendous amount of data to start learning and performing tasks with accuracy.

The process through which ANNs learn is based on learning paradigms—specific methods that dictate how these networks are trained and optimized. This study aims to understand how ANNs simulate human learning processes by diving particularly into reinforcement learning and supervised learning paradigms.

Key concepts:

- **Supervised Learning**

Supervised learning is the most widely used learning paradigm for ANNs. In supervised learning, the network is trained on labelled data, meaning that each input comes with its paired output. By assigning a network the learning of the right correlation between inputs and outputs, it will then be able to predict the output for new, unseen data.

For instance, in image classification, a neural network may be trained on a dataset where each image is labelled with its correct category (e.g., "galaxy" or "apple"). During the training, it makes predictions, compares them to the actual labels, and adjusts the parameters in order to minimize the error and thus maximize the accuracy.

- **Unsupervised Learning**

In unsupervised learning, instead of learning from input-output pairs, the algorithm tries to identify patterns, structures, or relationships within the data on its own. This allows the model to self-classify data by their found common patterns.

- **Reinforcement Learning**

Reinforcement Learning (RL) is a unique learning paradigm where an agent interacts with a dynamic environment by performing a series of actions through trial and error with delayed feedbacks (win/loss/draw), adjusting its strategy based on the rewards or penalties it receives for its specific actions. The interactions are often modelled as a Markov Decision Process (MDP).

MDP is the mathematical framework used to describe the RL environment. It consists of some main components:

- **States (S)**: The set of all possible situations the agent can be in. For example, in a game, each state might represent the current configuration of the game board.
- **Actions (A)**: The set of all possible actions the agent can take. Each action causes a transition from one state to another.
- **Rewards (R)**: After each action, the environment provides feedback in the form of a reward, which is a numerical value representing how desirable the resulting state is. The goal of the agent is to maximize its cumulative reward over time.

(GeeksforGeeks)

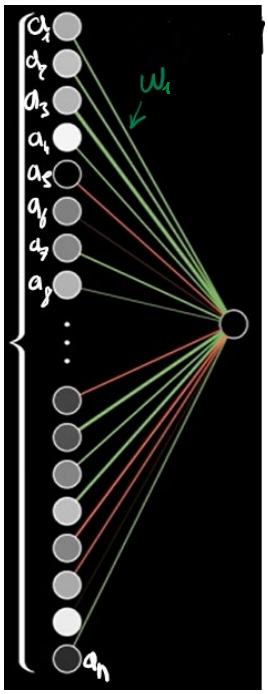
- **Discount Factor:** The discount factor is a number between 0 and 1 that determines how much future rewards are worth compared to immediate rewards. A value of 0 means the agent only cares about immediate rewards, while if it gets closer to 1, it means the agent is considering long-term rewards.

Reinforcement learning has been proven to be highly effective in applications that require decision-making, such as robotics, self-driving cars, and game AI, where the environment is more complex or where the desired outcome is not immediately clearly visible after each action.

Introduction to Artificial Neural Networks (ANNs):

Artificial Neural Networks (ANNs) consist of layers of nodes (or neurons), which are connected in a structure that processes information and generates an output, very similar to a biological neuron network.

The nodes of a layer are representing "activations values", a_i. The first layer represents the input values from a dataset we have, whereas the activation value of each node of the next layer is given by the equation explained step by step below. Also, each node of a layer is connected to every node from the previous and/or next layer. These connections are often called synapses and possess a "weight" which is a numerical value, w_i, that would kind of represent "the strength" of these.



Therefore, to find the activation value of a certain node (figure 1), we have to sum up the value of every node of the previous layer multiplied by their given weight : $a(x) = \sum_{i=1}^{n}(a_i w_i)$, with *n* being the number of previous connected nodes.

Figure 1-from https://www.youtube.com/watch?v=aircAruvnKk

However, by computing this equation we can find any value from an infinite range. But we want the range of the activation value to be finite. Because when the range of the activation values is finite, gradient-based training methods(explained later) are found to be more stable, so our model can perform with the best accuracy on a task.

So, there are a lot of different activation functions and the one that would return a finite range of values. For instance, the figure 3 below is called the “sigmoid function”(figure 2) and return values between [0, 1].

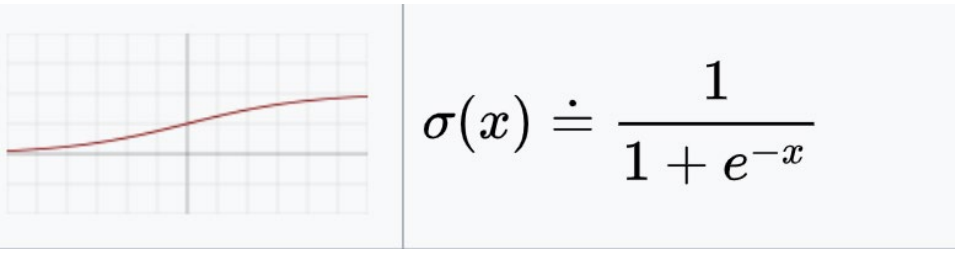


Figure 2- from https://en.wikipedia.org/wiki/Activation_function

Each activation function has its advantages and disadvantages. For instance, sigmoid is used principally for binary classification, which means that the number of nodes in the output layer is 2. So binary classification is used when we are dealing with only two different classes of outputs or only one.

For multi-class classification (when the number of nodes in the output layer ≥ 2), the SoftMax function is preferable (figure 3&4).

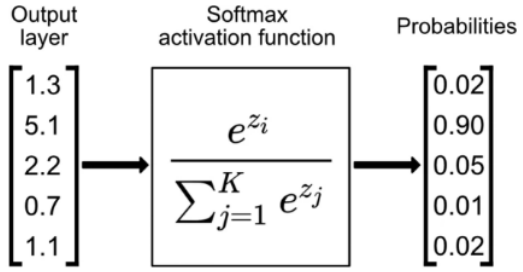
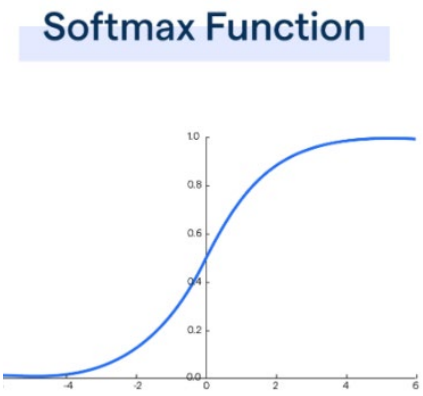


Figure 3- from https://www.singlestore.com/blog/a-guide-to-softmax-activation-function/



Range = [0, 1]

Figure 4- from https://botpenguin.com/glossary/softmaxfunction

On the top of that, deep artificial neural networks tend to require more than one activation function for a single network. Indeed, the SoftMax and sigmoid function are commonly used for the output layer, that means we might need a different activation function for the hidden layer(s).

The three most common functions are:

- Rectified Linear Unit (ReLU) : Default activation function for hidden layers in many deep learning models due to its computational simplicity and efficiency. Helps speeding up training and addressing the vanishing gradient problem. However, it can suffer from the "dying ReLU" problem, where the neurons can get stuck at zero.
- Leaky ReLU: function used in specific cases to prevent the "dying ReLU" problem.
- Exponential Linear Unit (ELU) : used to get smoother transitions and better gradient flow in deeper networks. It is also less likely to suffer from the vanishing gradients compared to ReLU and can output negative values.

Knowing this, we can change our equation $a(x) = \sum_{i=1}^{n}(a_i w_i)$, to a more accurate version $a(x) = g(\sum_{i=1}^{n}(a_i w_i))$, where g(x) is the chosen activation function.

A neuron activates based on how positive its weighted sum is. To control this, we add a bias, b_i. "In other words, this number tells you how high the weighted sum needs to be before the neuron starts getting meaningfully active." (Grant Sanderson)

Finally, we obtain $a(x) = g(\sum_{i=1}^{n}(a_i w_i + b_i))$.

Supervised Learning:

- **Forward propagation :**

Forward Neural Networks (FNNs) are the simplest ANN architecture used in supervised learning. In FNNs, information moves from input nodes through hidden layers to output nodes. Here is the implementation of an FNN (figure 5, 6 & 7) :

main.py

Share Run

```
import numpy as np

# Sigmoid activation function and its deriviative
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# Initialize parameters
np.random.seed(1)    # For reproducibility

input_layer_size = 2 # Number of input features (XOR)
hidden_layer_size = 2 # Number of neurons in the hidden layer
output_layer_size = 1 # Number of ouptut classes (binary)

# Weights initialization
weights_input_hidden = np.random.rand(input_layer_size,
    hidden_layer_size)
weights_hidden_output = np.random.rand(hidden_layer_size,
    output_layer_size)

# Bias initialization
bias_hidden = np.random.rand(hidden_layer_size)
```

Figure 5-from candidate's code

```
bias_output = np.random.rand(output_layer_size)

def forward_propagation(X):
    # Input to hidden layer
    z_hidden = np.dot(X, weights_input_hidden) + bias_hidden
    a_hidden = sigmoid(z_hidden)

    # Hidden layer to output
    z_output = np.dot(a_hidden, weights_hidden_output) +
        bias_output
    a_output = sigmoid(z_output)

    return a_output, a_hidden, z_hidden, z_output

# Example input
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
output, a_hidden, z_hidden, z_output = forward_propagation(X)
print("Output of forward propagation:\n", output)
```

Figure 6-from candidate's code

```
Output of forward propagation:
 [[0.62972271]
 [0.63125095]
 [0.63652524]
 [0.63766357]]

=== Code Execution Successful ===
```

Figure 7-from candidate's code

Explanation :

- The input_layer_size corresponds to the number of neurons in the input layer, or the number of features in our input data. Here, we gave 4 inputs of 2 features to our model, so the ANN returned 4 outputs of a single feature since output_layer_size = 1. The outputs here, represents the activation value of the neuron in the output layer given by the sigmoid function. Also, since there is a single neuron in the output layer, it indicates a binary classification.

- In this code, we chose hidden_layer_size = 2 arbitrarily for simplicity. But this number is often determined by Empirical testing : method of trying different sizes and evaluating network's performance, and by starting commonly with a number of hidden neurons that is between the size of the input and output layers.

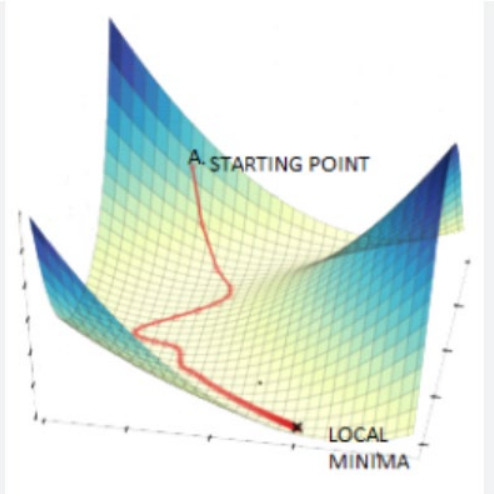
- Randomly initializing weights helps ensure that different neurons learn different features. If all weights were initialized to the same value, the neurons in each layer would learn the same features, which would make them redundant. Whereas biases are often initialized to zero or very small values, this helps ensure that each neuron starts learning from a neutral point.

- **Introduction to ANNs' training:**

Now, training an ANN involves the use of a gradient descent. The purpose of a gradient descent is to optimize the learning process by adjusting the parameters like weights and biases that will contribute to the minimization of the *cost* function. Every training task needs a cost function to measure how well the model is performing (learning) on this task. The cost function takes the loss values from all individual data points and returns a new number. This number represents the overall error of the model across the entire dataset. The loss values quantify the error between the predicted and the actual values. (EnjoyAlgorithms)

Once our ANN is given this number, it needs to adjust its parameters in order to learn and get better. First, it wants to find the input that minimizes the value of this cost function. For that, our model will start at a random input and look for the direction it should go to decrease the output of the function and find its minimum (as pictured in figure 8).

Figure 8- from https://www.slideshare.net/slideshow/feed-forward-back-propagationgradient-descent/240993251#18



In mathematics, the gradient of a function C is expressed as follow: $\nabla C(x, y, \dots)$ and gives the direction of the steepest ascent. However, we want the direction of the steepest descent, thus our equation should be : $-\nabla C(x, y, \dots)$.

In Calculus, this gradient can be represented as a vector : $\nabla C(x, y, ..) = \begin{bmatrix} \frac{\partial C}{\partial x} \\ \frac{\partial C}{\partial y} \\ \vdots \end{bmatrix}$.

Keeping in mind that each component of this gradient represents the weights. The parameters of our cost function are thus, $C(w_1, w_2, \dots, w_n)$.

In other words, the pseudo-code below summarizes what the model has to do :

#1 Initialise an input randomly

#2 Loop until convergence:

Compute $\nabla C(w1, w2, \dots)$

Go to $-\nabla C(w1, w2, \dots)$

#3 Return output

- **Backpropagation:**

Backpropagation, in Machine Learning, is just an algorithm for computing this complicated gradient. This algorithm can be summarized into those major steps :

1. Calculate the error at the output layer.
2. Compute the gradient (derivative) of the error with respect to weights
3. Iteratively move in the direction that minimizes the value of the cost function.
4. Update the weights and biases using the gradients and a learning rate.

For example, let's say we're developing a model to predict housing prices based on a single feature, such as the size of the house (in square feet). However, this model is not well trained yet.

1. **Initialization of weights and biases:**
    - Suppose you start with an initial guess for the weight (w) and bias (b) of the linear model. For simplicity, let's initialize them to zero.
2. **Forward Pass**:
    - Given the size of the house (let's say 1500 square feet) and the initial weight and bias, we make a prediction: Predicted Price $= w * Size + b$, with initial w and b both zero, the prediction is then zero.
3. **Calculate Loss**:
    - Suppose the actual price of the house is \$300,000. You calculate the loss (or error) using a cost function like Mean Squared Error (MSE): $Loss = \frac{1}{N}\sum_{i=1}^{N}(y_i - f(x_i))^2$, where *y* is the actual expected output and $f(x)$ is the predicted output by the model. This will give : $Loss = (300{,}000 - 0)^2 = 90{,}000{,}000{,}000$.
4. **Compute Gradients**:
    - To improve the model, you need to adjust the weight and bias. This is where backpropagation comes in. It involves computing the gradient of the cost function with respect to the weight and bias to see how changes in these parameters affect the loss.

$\frac{\partial Loss}{\partial w} = (y - f(x)) * Size = (300{,}000 - 0) * 1500 = 450{,}000{,}000$ and

$\frac{\partial Loss}{\partial b} = (y - f(x)) = 300{,}000$

5. **Update Weights and Biases**:
   - Using these gradients, you adjust the weights and biases in the direction that reduces the loss. If using a simple gradient descent algorithm with a learning rate (η) of 0.01 : $w = w - \eta * \frac{\partial Loss}{\partial w} = 0 - 0.1 * 45 * 10^7$

     and $b = b - \eta * \frac{\partial Loss}{\partial b} = 0 - 0.1 * 300{,}000$

6. **Repeat**:
   - You repeat the forward pass, loss calculation, backpropagation, and the recursive weight update steps iteratively. Each iteration should ideally bring the model's predictions closer to the actual house prices, by reducing the loss.

Pseudo-code algorithm for backward propagation:

1. Initialize weights randomly
2. Loop until convergence:
3. Compute gradient: $\frac{\partial C(W)}{\partial w}$
4. Update weights, $W \leftarrow W - \eta \frac{\partial C(W)}{\partial w}$
5. Return weights

In other words, the backward propagation algorithm necessarily involves forward propagation, our model will then alternate those two processes until it reaches the final desired output, in order to learn. Artificial neural networks will learn through backpropagation, improving the model's prediction, and by repeating this process, achieving more accurate predictions as the loss decreases. This algorithm is crucial in developing our ANN, so that it can generalize well to new, unseen, data and thereby enhancing their practical utility.

However, some models manage to perform extremely well on the training data, in fact these models are going to learn the training data too well and memorized them instead of learning their underlying patterns, which will negatively impact the model's performance on new, unseen data, due to the poor generalization it will be able to do. Multiple causes are behind what we call this, **overfitting**, but multiple solutions exist.

Solutions:

- **Regularization:** Techniques like L1/L2 regularization add a penalty for large weights, helping to prevent overfitting.
- **Cross-Validation:** Splitting data into multiple training and validation sets to ensure the model performs well across different subsets.
- **Dropout:** Randomly setting some neurons to zero during training to prevent co-adaptation of hidden units.
- **Simplify the Model:** Reduce the complexity of the model by decreasing the number of layers or neurons.
- **Hyperparameter Tuning:** Adjust parameters like the learning rate or the number of epochs to find an optimal configuration.
- **Early Stopping (figure 9):** Monitoring the model's performance on a validation set and stopping training when performance starts to degrade on new, unseen data.

(David Chuan-En Lin)



Figure 9-from MIT open courseware

The following data were found by the implementation of the code in the appendix A.1.

This will help us to understand when and why our models overfit.

The programming language chosen is python and the code was implemented via the Jupyter Notebook. Our code was returning a graph and by that graph we could deduce the epoch at which the model started to overfit, as shown in in the appendix A.1 figure 11.

Table n°1 : Experimental data

	Number of epochs	Number of samples	Model size (number of hidden neurons)	Learning rate	Epoch at which the model starts overfitting
Model n°1	1000	500	50	0.01	~300
Model n°2	1000	5000	50	0.01	~600
Model n°3	5000	1000	100	0.001	~2000
Model n°4	5000	10000	100	0.001	~3000
Model n°5	3000	2000	100	0.01	~500
Model n°6	3000	2000	10	0.01	~1400
Model n°7	6000	5000	100	0.001	~1100
Model n°8	6000	5000	100	0.01	~800

Table 2: Experimental data

	Train and Test loss before overfitting	Train loss after overfitting	Test loss after overfitting	Train accuracy after overfitting	Test accuracy after overfitting
Model n°1	Decreasing	Decreasing	Increasing	~100%	Declining from ~80% to ~60%
Model n°2	Decreasing	Decreasing	Increasing slightly and plateau	~98%	Plateau at ~90%
Model n°3	Decreasing	Decreasing	Increasing	~100%	Declining from ~85% to ~50%
Model n°4	Decreasing	Decreasing	Increasing slightly	~99%	Declining from ~95% to ~80%
Model n°5	Decreasing	Decreasing	Increasing	~100%	Declining from ~70% to ~40%
Model n°6	Decreasing	Decreasing	Increasing slightly and plateau	~98%	Plateau at ~92%

Model n°7	Decreasing	Decreasing	Increasing	~100%	Declining from ~90% to ~60%
Model n°8	Decreasing	Decreasing	Increasing significantly	~100%	Declining from ~80% to ~40%

Analysis:

From table 1:

- Number of samples: By comparing the model n°1 with the model n°2, and the model n°3 with n°4, we can observe that the overfitting epochs start later when given more samples. This is because the model is given more data to train on, and therefore has learned a bit more.
- Number of hidden neurons: By comparing the model n°5 and the model n°6, we can see that the overfitting epochs start later with fewer number of neurons. In fact, when the model is simpler, it has fewer neurons to learn the underlying patterns of the data. Consequently, the model generalizes better to unseen data for a longer period during testing.
- Learning rate : By comparing the model n°7 and the model n°8, we can suggest that when the learning rate is lower, the model learns slower during training data, and thus, doesn't have the time to learn the noise in the data and consequently performs better on the test data. Therefore, the overfitting epoch is delayed.

From table 2:

- For all models, training loss decreases steadily, which is expected since the gradient descent algorithm is designed to minimize this loss. This indicates that each model is learning to reduce the error on the training data.
- As expected for all models test loss starts to increase. This pattern suggests that the models start memorizing specific details in the training data, leading to poorer performance on new, unseen data.
- Training accuracy rises consistently for all models, reaching close to 98-100% after overfitting. This indicates that each model becomes highly accurate at predicting outcomes on the training data, a common result when models start overfitting by

memorizing rather than generalizing. And therefore, the test accuracy either drops or stagnates.

- The table reflects classic signs of overfitting: a divergence between training and test loss and accuracy after a certain point. This means that while the models are optimized on training data, their real-world performance (test accuracy) suffers.
- To improve these models, the techniques discussed earlier could help reduce overfitting and enhance generalization to unseen data.

Limitations:

Aspect	**Explanation**	**Potential Improvement**
Noisiness of datasets	Noise in the data can lead models to overfit on random variations, reducing their ability to generalize.	Use practical and less noisy data to allow models to capture meaningful patterns.
Random datasets	Completely random data lacks real-world relevance, limiting insights on the model's generalization to practical tasks.	Use datasets with structure or real-world significance to assess generalization on meaningful data.
Time complexity	Some models take over 5 minutes to train, which is inefficient given the small dataset size and random nature of data.	Frameworks like PyTorch or TensorFlow are better suited for handling complex models and deep-learning tasks in general.
Computational Environment	Limited framework (like Jupyter Notebook) can affect model performance and efficiency.	Same as time complexity
Models' configuration	Multi-layer models are often better suited for complex patterns. Limited layer depth hinders the model's ability to learn nuanced relationships.	Implement multi-layer models and optimize hyperparameters to improve generalization.

Experimental Scope and Statistical Power	Insufficient number of models reduce the robustness of findings, making it harder to draw statistically significant conclusions.	Implement and train a larger variety of models to provide better evidence of trends and patterns.

As a concluding statement, while ANNs rely on mathematical algorithms to process and learn patterns, human cognition operates through electrochemical reactions via the brain. This difference impacts flexibility and generalization; ANNs lack the full spectrum of adaptability seen in human thought and response. This one of the limitations ANN are facing in comparison with human cognition.

Reinforcement learning (RL):

- **Q-Learning:**

Among the several techniques used in RL such as Policy gradient methods or even Actor-Critic methods; Q-Learning provides a hands-on approach, making it easier to demonstrate with coding examples like a simple grid environment. The code in the appendix A.2 allows us to understand the decision-making process of our model, although the analysis is intended to be straightforward.

The goal of our model is to reach the desired position inside a small grid, by getting rewards over time if it succeeds. The implementation of the code provides us with the following graph:

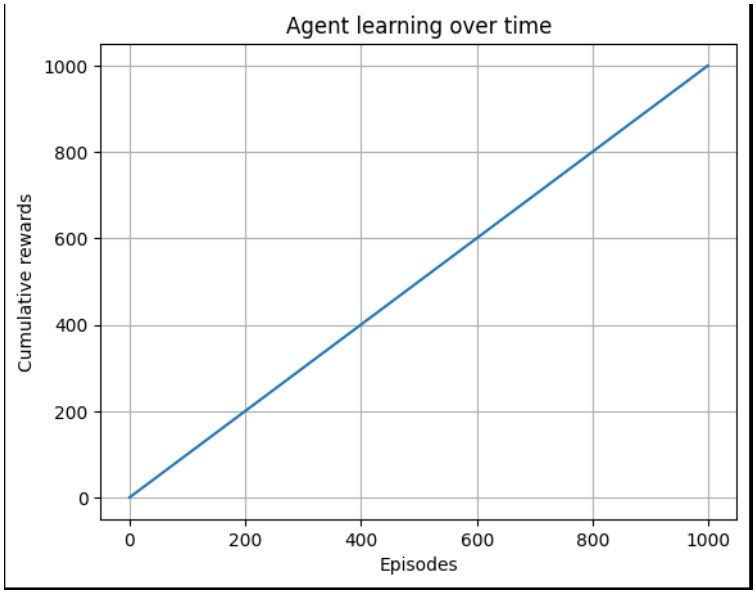


Figure 10- graph obtained with the code in appendix A.2

This straight line gives us an easy interpretation of our model:

- The agent is consistently learning to obtain rewards over time.
- It suggests that the agent is reaching the goal state in every episode

This is certainly due to the simple environment our model is performing in. Also, the environment may not have much stochasticity (randomness), meaning the agent's moves and results are predictable, leading to linear accumulation of rewards.

The exploration rate might also be too high given the small grid we have, causing our model to explore everything early on, therefore learning the optimal policy from the beginning and as a result, it consistently achieves the maximum possible reward in each episode thereafter.

In a more complex environment, the cumulative reward graph will be a non-linear shape, reflecting slower learning at first and improving over time. The agent would likely face more challenges initially, but after many episodes, it will still eventually converge towards a near-linear steady increase once it has learned the optimal policy.

Given what we talked about so far, we can explain why ANNs perform well in structured games like chess but struggle in RPGs:

Chess is a rule-bounded environment with predictable outcomes. In chess, every possible move and consequence can be mathematically modelled, allowing ANNs to calculate optimal moves and learn strategies efficiently through reinforcement learning.

RPGs, however, often involve complex environments with unpredictable interactions, narrative choices, and real-time decision-making. Unlike chess, where all information is available and clear, RPGs require nuanced understanding, memory, and adaptability to a wide variety of scenarios that change dynamically. These elements are hard to quantify mathematically and cannot easily be reduced to fixed states or rewards. This is the extent to how effectively ANNs simulate human learning processes.

Conclusion:

Artificial Neural Networks (ANNs) have come a long way in simulating human-like learning processes, inspired from the structure of the human brain. From image recognition to natural language processing and autonomous decision-making, the study of ANNs illustrates how effectively they simulate human cognitive processes.

In learning, emerging methods have allowed ANNs to reach the human brain's ability whether it be in terms of accuracy or efficiency. In **supervised learning**, ANNs simulate the human ability to learn from labelled examples, mimicking how humans acquire knowledge through instructions. We saw that they sometimes tend to memorize the training data instead of learning their underlying patterns, much like how students usually tend to prepare for important exams.

Unsupervised learning reflects the brain's capacity to make sense of data without explicit guidance, much like how humans understand their surroundings through observation and inference. **Reinforcement learning** mirrors the way humans learn through trial and error, with instant feedbacks, consistently adjusting their strategies in response to the rewards and punishments they receive, allowing them to adapt to the dynamic environment they are in.

These learning paradigms are not used separately but rather used together to complement each other in order to get the desired result on a task, just as human learning integrates multiple cognitive processes to get a global understanding of their environment. New emerging methods are also to consider, such as multi-task learning. This approach forces the model to learn shared representations that are useful across tasks, improving generalization and reducing overfitting. In supervised learning, this could involve training a neural network to predict multiple attributes of an image (e.g., object type, colour, and size). In reinforcement learning, multi-task learning allows agents to learn policies that generalize across different environments or tasks. For example, a robot trained via multi-task RL could learn to perform multiple actions (grasping, navigating, avoiding obstacles) within the same training framework. Multi-task learning is one of foundations of human knowledge, which helps AI to get closer and closer to human's cognitive abilities.

However, despite their impressive advances, ANNs are not perfect models of human cognition. While they can learn from a vast amount of data and perform specific tasks at higher levels, they often lack reasoning ability, learning stability, and general understanding that define true human intelligence. Furthermore, unlike humans, who can learn from unlimited external interactions and continuous feedback, ANNs require a huge amount of data to function effectively.

But in the end, with perpetual research in mathematical algorithms, combined with advances in Machine Learning, we continue to refine and evolve these artificial neural networks and move closer to building intelligent systems that will not only simulate but also surpass human cognition, opening the way for AI-driven innovations. Even though we should not start a comparing war between AI abilities and human abilities, but rather enjoy seeing them serve one another for all type of tasks.

Bibliography

EnjoyAlgorithms. (n.d.). *Introduction to loss and cost function in machine learning*. Retrieved October 19th, 2024, from EnjoyAlgorithms: https://www.enjoyalgorithms.com/blog/loss-and-cost-functions-in-machine-learning

GeeksforGeeks. (n.d.). *Markov decision process*. Retrieved August 15th, 2024, from GeeksforGeeks: https://www.geeksforgeeks.org/markov-decision-process/

Grant Sanderson, 3. (n.d.). *"But what is a neural network | Chap 1, deep learning"*. Retrieved october 17th, 2024, from Youtube: https://www.youtube.com/watch?v=aircAruvnKk

Lin, D. C.-E. (n.d.). *8 simple techniques to prevent overfitting*. Retrieved August 17th, 2024, from Towards Data Science: https://towardsdatascience.com/8-simple-techniques-to-prevent-overfitting-4d443da2ef7d

Appendix A.1 : Simple neural network implementation

```
""" Here is the implementation of a simple neural network
that learns the following classification rule :
    y = 1 if x1 + x2 > 0
    y = 0 if x1 + x2 <= 0
"""

import numpy as np # type: ignore
import matplotlib.pyplot as plt # type: ignore

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

def mse_loss(y, prediction):
    return np.mean((y - prediction) ** 2)

class SimpleNN:

    def __init__(self, input_size, hidden_size, output_size, learning_rate):

        self.learning_rate = learning_rate

        # Random weight initialization
        self.weights_input_hidden = np.random.randn(input_size, hidden_size) * 0.1
        self.weights_hidden_output = np.random.randn(hidden_size, output_size) * 0.1

        # Bias initialization to zero
        self.bias_hidden = np.zeros((1, hidden_size))
        self.bias_output = np.zeros((1, output_size))

    def forward(self, X):

        self.hidden_input = np.dot(X, self.weights_input_hidden) + self.bias_hidden
        self.hidden_output = sigmoid(self.hidden_input)

        self.output_input = np.dot(self.hidden_output, self.weights_hidden_output) + self.bias_output
        self.output = sigmoid(self.output_input)

        return self.output

    def backward(self, X, y, output):

        N = int(samples * ratio)
        output_error = output - y # comes from the derivative of the mse

        output_delta = (2 / N) * output_error * sigmoid_derivative(output)
        output_gradient = np.dot(self.hidden_output.T, output_delta)
        output_bias_gradient = np.sum(output_delta, axis=0)
        hidden_error = np.dot(output_delta, self.weights_hidden_output.T)
        hidden_delta = hidden_error * sigmoid_derivative(self.hidden_output)
        hidden_gradient = np.dot(X.T, hidden_delta)
        hidden_bias_gradient = np.sum(hidden_delta, axis=0)
```

```
        self.weights_hidden_output -= self.learning_rate * output_gradient
        self.weights_input_hidden -= self.learning_rate * hidden_gradient
        self.bias_output -= self.learning_rate * output_bias_gradient
        self.bias_hidden -= self.learning_rate * hidden_bias_gradient

    def train(self, X_train, y_train, X_test, y_test, epochs):

        train_losses = []
        test_losses = []

        for epoch in range(epochs):

            # Predicting
            output = self.forward(X_train)

            # Learning
            self.backward(X_train, y_train, output)

            # Evaluation
            output_train = self.forward(X_train)
            output_test = self.forward(X_test)
            train_loss = mse_loss(y_train, output_train)
            test_loss = mse_loss(y_test, output_test)
            train_losses.append(train_loss)
            test_losses.append(test_loss)
            if epoch % 100 == 0:
                print("Epoch:", epoch, "Train Loss:", train_loss,"Test Loss:", test_loss)

        return train_losses, test_losses

    def predict(self, X):
        probabilities = self.forward(X)
        predictions = []
        for value in probabilities:
            if value[0] >= 0.5:
                predictions.append(1)
            else:
                predictions.append(0)
        return predictions

# Parameters
samples = 500
nb_input_nodes = 2 # do not change for the very principle of the rule
nb_hidden_nodes = 5
nb_output_nodes = 1 # do not change for the very principle of the rule
learning_rate = 0.1
ratio = 0.7

# Data initialization
np.random.seed(42)
X = np.random.randn(samples, nb_input_nodes)
y = []
```

```python
for i in range(samples):
    if X[i, 0] + X[i, 1] > 0:
        y.append([1])
    else:
        y.append([0])

# Train / Test split (ex : 70% / 30%)
split = int(ratio * samples)
y = np.array(y)
X_train = X[:split]
X_test = X[split:]
y_train = y[:split]
y_test = y[split:]

# init
nn = SimpleNN(nb_input_nodes,nb_hidden_nodes,nb_output_nodes,learning_rate)

predictions = nn.predict(X_test)
correct = 0
for i in range(len(y_test)):
    if predictions[i] == y_test[i, 0]:
        correct += 1
accuracy = correct * 100 / len(y_test)
print(f"Accuracy before training: {accuracy}%")

# train
train_losses, test_losses = nn.train(X_train,y_train,X_test,y_test,epochs=2000)

# final output
predictions = nn.predict(X_test)
correct = 0
for i in range(len(y_test)):
    if predictions[i] == y_test[i, 0]:
        correct += 1
accuracy = correct * 100 / len(y_test)
print(f"Accuracy after training: {accuracy}%")

plt.plot(train_losses, label="Train Loss")
plt.plot(test_losses, label="Test Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE")
plt.title("Evolution of the cost function")
plt.legend()
plt.show()
```

✓ 0.4s Python

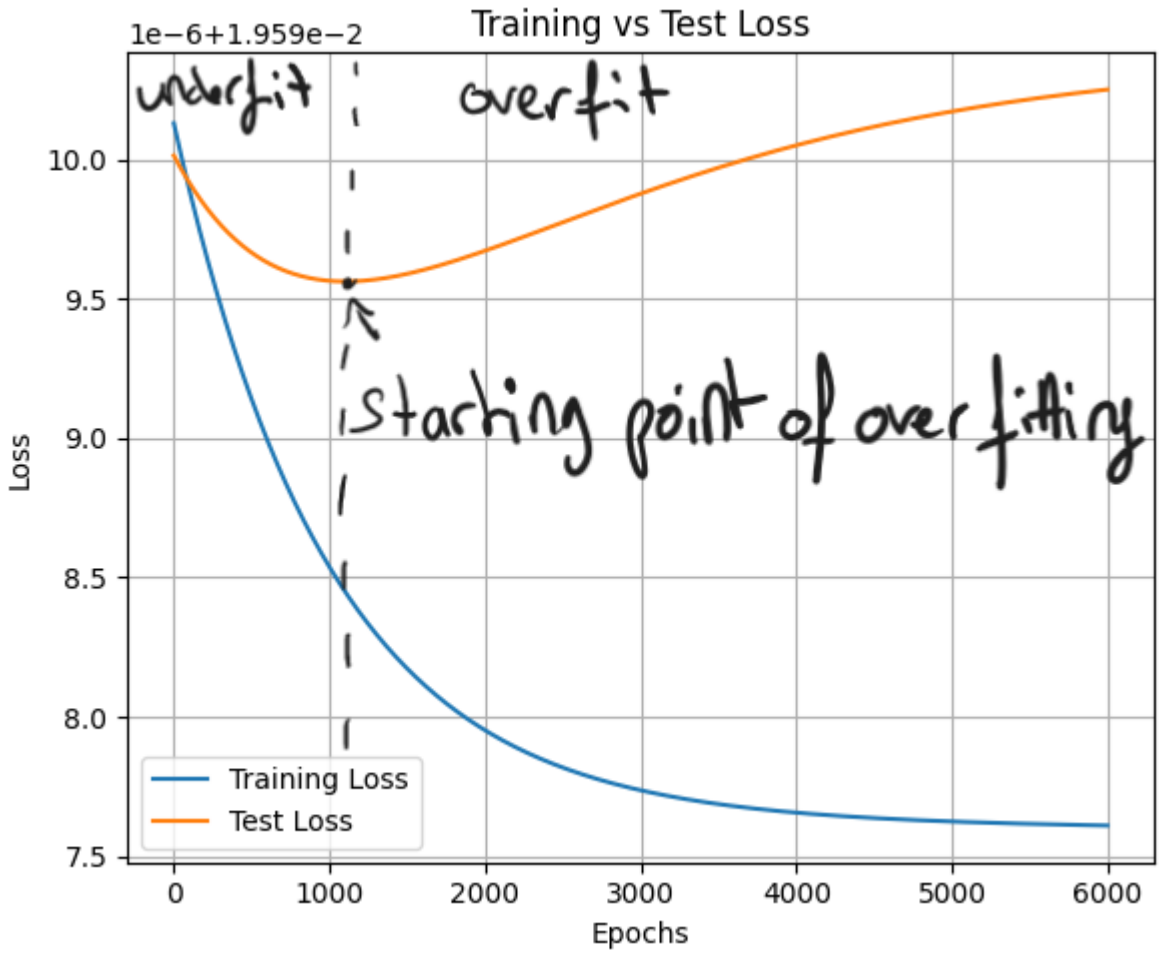


Figure 11-graph obtained with the code above for model n°7

Appendix A.2 : Q-Learning implementation

```
import numpy as np
import random
import matplotlib.pyplot as plt

# Grid environment settings
grid_size = (5, 5) # 5x5 grid
goal_state = (4, 4) # goal position
num_episodes = 1000 # Number of episodes

# Initialize the Q-Table
Q = np.zeros((*grid_size, 4)) # 4 actions : up, down, left, right

# Parameters
alpha = 0.1 # Learning rate
beta = 0.9 # Discount factor
gamma = 0.1 # Exploration rate

def get_next_state(state, action):
    """Return the next state given the current state and action"""
    if action == 0: # Up
        next_state = (max(0, state[0] - 1), state[1])
    elif action == 1: # Down
        next_state = (min(grid_size[0] - 1, state[0] + 1), state[1])
    elif action == 2: # Left
        next_state = (state[0], max(0, state[1] - 1))
    elif action == 3: # Right
        next_state = (state[0], min(grid_size[1] - 1, state[1] + 1))
    return next_state

def get_reward(state):
    if state == goal_state:
        return 1 # Reward for reaching the goal
    return 0 # No reward otherwise
```

```
def q_learning(num_episodes):
    """Train the agent using Q-Learning"""
    rewards = [] # Track rewards for later analysis
    for _ in range(num_episodes):
        state = (0, 0) # Start at the top-left corner
        total_reward = 0
        while state != goal_state:
            if random.uniform(0, 1) < gamma: # Exploration
                action = random.randint(0, 3) # Random action
            else:
                action = np.argmax(Q[state]) # Best known action
            next_state = get_next_state(state, action)
            reward = get_reward(next_state)

            # Q-Learning update rule
            Q[state][action] += alpha*(reward + beta*np.max(Q[next_state]) - Q[state][action])
            state = next_state
            total_reward += reward
        rewards.append(total_reward)
    return rewards

# Train the agent
rewards = q_learning(num_episodes)

# Analyze and visualize the rewards
plt.plot(np.cumsum(rewards))
plt.xlabel('Episodes')
plt.ylabel('Cumulative rewards')
plt.title('Agent learning over time')
plt.grid()
plt.show()
```